

Постановка задачи.

Дана фигура на плоскости согласно вариантам. Фигура описывается индивидуальными геометрическими свойствами и общими оформительскими свойствами: цвет, видимость. У фигуры имеются характеристики: периметр, площадь, ограничивающая область. Область размещения фигур в плоскости ограничена экстендами, за которые фигура не должна выходить.

Необходимо разработать:

- классы для описания положения «Location» и ограничивающей области «Clip» в плоскости;
- статический класс «Geometry» для хранения общих констант и методов проверки различных ограничений на размещение фигур в плоскости;
- класс геометрического примитива «Primitive» для хранения и редактирования оформительских свойств фигуры как наследника от статического класса «Geometry»;
- класс примитивной фигуры – точки «Point» как наследника от классов «Location» и «Primitive»;
- класс фигуры согласно варианту «Figure» как наследника от класса «Point» с описанием специфических свойств и методов фигуры;
- наборы конструкторов для создания экземпляров каждого класса различными способами (дефолтный, копирующий, параметрический);
- методы для изменения свойств и вычисления характеристик фигуры;
- интерфейс для отображения и изменения всех свойств фигуры.

Интерфейс и классы реализуются в одном модуле. Для редактирования фигуры разработать функцию ModifyFigure(), которая должна получать ссылку на экземпляр фигуры и предоставлять интерактивный консольный интерфейс для работы с ним.

Вариант 23. Фигура: правильный шестиугольник.

Описание свойств фигуры:

Особенность правильного шестиугольника — равенство его стороны и радиуса описанной окружности ($R = t$).

Периметр: сумма сторон шестиугольника

$$P = 6R$$

Радиус вписанной окружности:

$$r = \frac{\sqrt{3}}{2} R$$

Площадь правильного шестиугольника:

$$S = \frac{3\sqrt{3}}{2} R^2$$

Ограничивающая область:

$$E = (x_0 - r, y_0 - R)_{min} \rightarrow (x_0 + r, y_0 + R)_{max}$$

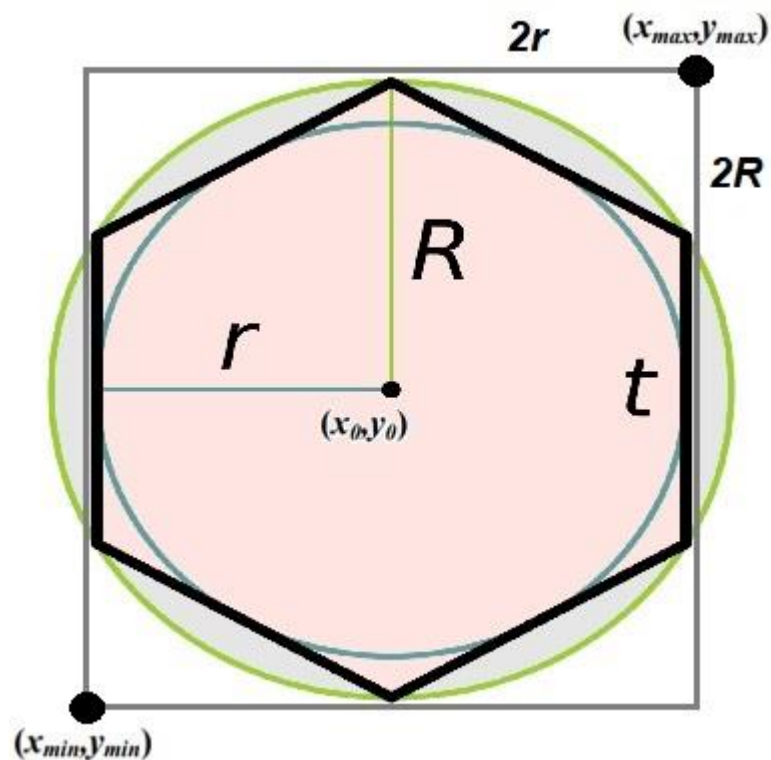


Рисунок 1 – Фигура

Диаграмма иерархии классов:

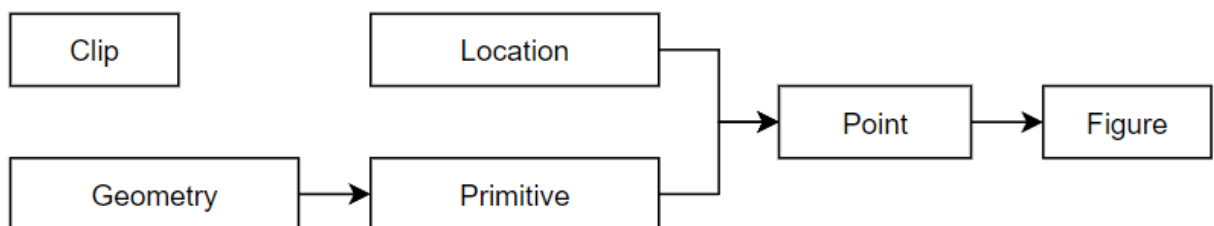


Рисунок 2 – Диаграмма иерархии классов

Тестовый план:

Таблица 1 – Исходные данные и результаты

№	Исходные данные	Результаты
1	X = 1, Y = 1, R = 1	P=6, r = 0.866025, S=2.59808, E=(0.133975;0)-(1.86603;2)
2	X = 2, Y = 2, R = 2	P=12, r = 1.73205, S=10.3923, E=(0.267949;0)-(3.73205;4)
3	X = -1, Y = -1, R = -5	P=30, r = 4.33013, S=64.9519, E=(-5.33013;-6)-(3.33013;4)

Листинг программы:

```

#include <math.h>
#include <iostream>
using namespace std;
/* Класс реализует позицию (x,y) на плоскости */
class Location {
public:

```

```

    double x; // X-координата
    double y; // Y-координата
    Location() : x(0), y(0) { }
    Location(const Location& obj) : x(obj.x), y(obj.y) {}
    Location(double xx, double yy) : x(xx), y(yy) {}
};

/* Класс реализует ограничивающую область на плоскости */
class Clip {
public:
    Location min; // Левая нижняя граница
    Location max; // Правая верхняя граница
    Clip() : min(), max() {}
    Clip(const Clip& obj) : min(obj.min), max(obj.max) {}
    Clip(double xn, double yn, double xk, double yk)
    {
        if (xn < xk) {
            min.x = xn;
            max.x = xk;
        }
        else {
            min.x = xk;
            max.x = xn;
        }
        if (yn < yk) {
            min.y = yn;
            max.y = yk;
        }
        else {
            min.y = yk;
            max.y = yn;
        }
    }
    double sizeX() const { return (max.x - min.x); }
    double sizeY() const { return (max.y - min.y); }
};

/* Класс реализует геометрические константы и функции проверки
(базовый статический класс для всех примитивов) */
class Geometry {
public:
    static const double pi;
    static const double pi2;
    static const double extent;

```

```

static double accurateExtent(double value)
{
    return ((value < -extent) ? -extent : (value > extent) ? extent : value);
}

};

const double Geometry::pi = 3.14159265358979323846;
const double Geometry::pi2 = 6.28318530717958647693;
const double Geometry::extent = 1E6;

class Primitive : public Geometry {
private:
    bool visible;
    unsigned char color;
public:
    int getColor() const { return int(color); }
    void setColor(int tint)
    {
        color = unsigned char((tint < 0) ? 0 : (tint > 255) ? 255 : tint);
    }
    bool isVisible() const { return visible; }
    void setShow() { visible = true; }
    void setHide() { visible = false; }
    Primitive() : visible(false), color(0) { }
    Primitive(const Primitive& obj) : visible(obj.visible), color(obj.color) {}
};

/* Класс реализует цветную точку на плоскости (родитель для фигуры) */
class Point : public Primitive, public Location {
public:
    Point(): Primitive(), Location() { }
    Point (const Point& obj): Primitive (obj), Location (obj) {}
    Point(double xx, double yy, int tint)
    {
        setX(xx);
        setY(yy);
        setShow();
        setColor(tint);
    }
    double getX() const { return x; }
    double getY() const { return y; }
    Location getPosition() const { return Location (x, y); }
    virtual void setX (double xx) { x = accurateExtent (xx); }
    virtual void setY (double yy) { y = accurateExtent(yy); }
    virtual Clip getClipBox() const { return Clip (x, y, x, y); }

```

```

};

/* Класс реализует фигуру "правильный шестиугольник" (наследник для
точки) */
class Figure : public Point {
private:
    double R; // Радиус описанной вокруг шестиугольника окружности
public:
    Figure() : Point(), R(0) { }
    Figure(const Figure& obj) : Point(obj), R(obj.R) { }
    Figure(double xc, double yc, double rd, int tint)
    {
        setColor(tint);
        setShow();
        setRadius(rd);
        setY(yc);
        setX(xc);
    }
    double getRadius() const { return R; }
    void setRadius(double r) { R = ((r < 0) ? -r : r); }
    double getInnerRadius() const { return R * (sqrt(3) / 2); } // Радиус
    вписанной в шестиугольник окружности
    double getLength() const { return R * 6; }
    double getSquare() const { return ((3 * sqrt(3)) / 2) * (R * R); }
    virtual void setX(double xx)
    {
        x = (((xx - R) < -extent) ? (-extent + R) :
            ((xx + R) > +extent) ? (+extent - R) : xx);
    }
    virtual void setY(double yy)
    {
        y = (((yy - R) < -extent) ? (-extent + R) :
            ((yy + R) > +extent) ? (+extent - R) : yy);
    }
    virtual Clip getClipBox() const
    {
        return Clip(x - getInnerRadius(), y - R, x + getInnerRadius(), y + R);
    }
};

void ErrorValue()
{
    std::cin.clear(); std::cin.sync();
    std::cout << "Ошибка: некорректное значение" << endl;
}

```

```

}

void ModifyFigure(Figure& Fig)
{
    char ch = 'P';
    do {
        switch (ch)
        {
            case 'P':
            case 'p':
                std::cout << endl << "Свойства фигуры:"
                    << endl << "Центр = (" << Fig.getX() << ";" <<
Fig.getY() << ")"
                    << endl << "Длина стороны шестиугольника = " <<
Fig.getRadius()
                    << endl << "Радиус вписанной окружности = " <<
Fig.getInnerRadius()
                    << endl << "Площадь = " << Fig.getSquare()
                    << endl << "Периметр = " << Fig.getLength()
                    << endl << "Видимость = " << ((Fig.isVisible()) ? "On" :
"Off")
                    << endl << "Номер цвета = " << Fig.getColor()
                    << endl;
                {
                    Clip area = Fig.getClipBox();
                    std::cout << "Область = (" << area.min.x << ";" <<
area.min.y
                        << ")-(" << area.max.x << ";" << area.max.y << ")"
                    << endl
                        << "Размер = [" << area.sizeX() << "x" <<
area.sizeY() << "]"
                        << endl;
                }
                break;
            case 'X':
            case 'x':
                {
                    double value;
                    std::cout << endl << "Введите X-координату:>";
                    std::cin >> value;
                    if (!std::cin.fail()) Fig.setX(value);
                    else ErrorValue();
                }
                break;
        }
    }
}

```

```

case 'Y':
case 'y':
{
    double value;
    std::cout << endl << "Введите Y - координату:>";
    std::cin >> value;
    if (!std::cin.fail()) Fig.setY(value);
    else ErrorValue();
    break;
}
case 'R':
case 'r':
{
    double value;
    std::cout << endl << "Введите радиус [R > 0]:>";
    std::cin >> value;
    if (!std::cin.fail()) Fig.setRadius(value);
    else ErrorValue();
}
break;
case 'T':
case 't':
{
    int tint;
    cout << endl << "Введите цвет [0..255]:>";
    cin >> tint;
    if (!cin.fail()) Fig.setColor(tint);
    else ErrorValue();
}
break;
case 'V':
case 'v':
    cout << endl << "Видимость [N-on | F-off]:>";
    cin >> ch;
    if (ch == 'N' || ch == 'n') Fig.setShow();
    else if (ch == 'F' || ch == 'f') Fig.setHide();
    else ErrorValue();
    break;
default: cout << "Ошибка: некорректная операция" << endl;
}
cout << endl << "(T) Изменить цвет"
    << endl << "(R) Изменить радиус"
    << endl << "(V) Изменить видимость"
    << endl << "(X) Изменить X-координату"

```

```

        << endl << "(Y) Изменить Y-координату"
        << endl << "(P) Печатать все свойства"
        << endl << "(Q) Выход";
    cout << endl << "Выберите пункт:>";
    cin >> ch;
}

while (ch != 'Q' && ch != 'q');
}

int main()
{
    setlocale(LC_ALL, "Russian");
    Figure figure = Figure(1, 1, 1, 255);
    // Figure figure = Figure(2, 2, 2, 255);
    // Figure figure = Figure(-1, -1, -5, 255);
    ModifyFigure(figure);

    return 0;
}

```

Результат выполнения программы (снимок экрана):

```

C:\Users\minsk\source\repos\C Sharp cmd\OOTPiSP\x64\Debug\KR_1_cpp.exe
Свойства фигуры:
Центр = (1;1)
Длина стороны шестиугольника = 1
Радиус вписанной окружности = 0.866025
Площадь = 2.59808
Периметр = 6
Видимость = On
Номер цвета = 255
Область = (0.133975;0)-(1.86603;2)
Размер = [1.73205x2]

(T) Изменить цвет
(R) Изменить радиус
(V) Изменить видимость
(X) Изменить X-координату
(Y) Изменить Y-координату
(P) Печатать все свойства
(Q) Выход
Выберите пункт:>_

```